

**FEDERAL UNIVERSITY OF RIO DE JANEIRO
POLYTECHNIC SCHOOL
DEPARTMENT OF ELECTRONIC AND COMPUTER
ENGINEERING**

UNDERGRADUATE PROJECT PROPOSAL

Student: Matheus Hoffmann Fernandes Santos
hoffmann@poli.ufrj.br

Advisor: To be defined, Dr.

Co-advisor: Pedro Veras Guimarães, Dr. (LabECO, Department of Mechanical
Engineering, UFSC)

1. TITLE

Reducing simulation and forecast time of the WAVEWATCH III wave model in the operational cycle of ReNOMO at LabECO/UFSC: compile options, modern Fortran and GPU feasibility.

2. EMPHASIS

Computing / High-performance scientific computing.

3. THEME

Third-generation spectral wave models solve the wave action balance equation on a spatial grid and a spectral grid of frequencies and directions, integrating the physics of wind input, nonlinear interactions and dissipation at every point and every time step. WAVEWATCH III (WW3), developed at NOAA/NCEP, is the reference model of this class and the one adopted by most operational centres, research groups and industry users worldwide (NOAA-EMC, 2026a; The WAVEWATCH III Development Group, 2019). It is written in Fortran, parallelised with MPI and OpenMP, and its behaviour is fixed at compile time by a *switch file* that selects the physics packages, the propagation scheme and the I/O options.

The Laboratório de Engenharia e Ciências Oceânicas (LabECO) of the Department of Mechanical Engineering at the Universidade Federal de Santa Catarina (UFSC) runs WW3 as part of its contribution to the Rede Nacional de Observação e Monitoramento Oceânico (ReNOMO), a national network funded by the CNPq/MCTI/Finep 062/2022 call that integrates ocean observation, quality con-

trol and operational product delivery for the South Atlantic and the Brazilian coast (LabECO/UFSC, 2026; UFSC, 2026). In an operational setting the cost of a forecast is measured in wall-clock time: the time between the arrival of the atmospheric forcing and the delivery of wave products bounds how often the model can be run, at which resolution, on how many nested domains, and how many ensemble members or sensitivity runs fit into a day of computing.

The theme of this project is therefore the systematic reduction of the wall-clock time of WW3 in the configuration used by LabECO, treated as an engineering problem with measurable outcomes: a reproducible benchmark of the current run, a profile that says where the time goes, and a ladder of optimisations of increasing depth, from build options that cost nothing to change, through the run configuration, to targeted refactoring of the hottest routines in modern Fortran, and finally to a feasibility study of executing the model on a data-centre GPU (NVIDIA H100). Each rung is measured on the laboratory’s own configuration and accepted only if the results remain within a tolerance agreed with the scientists, so that the operational chain never trades correctness for speed.

4. SCOPE

The work is limited to the single-grid driver of WW3 (`ww3_shel`) in the configuration operated by LabECO: its grid, its switch file, its forcing and its output list. That configuration is frozen at the start of the project and documented in a configuration record, together with the pinned source revision of the model, so that every measurement refers to the same code and the same case.

Four rungs are in scope, attempted in order and each gated by a comparison against the frozen reference run:

1. **Build options.** Compiler, optimisation flags, the switch file (in particular the choice between shared-memory, MPI and hybrid builds, and the NetCDF-4 output path), and the MPI and OpenMP layout on the available nodes. This rung changes no source code and is expected to be bit-for-bit reproducible against the reference or, when floating-point reordering is involved, within round-off.
2. **Run configuration.** Domain decomposition, time steps allowed by the CFL condition, output frequency and output fields, restart strategy, and forcing

interpolation. This rung changes namelist inputs only.

3. **Targeted modern-Fortran refactoring.** Rewriting of the routines that the profile identifies as dominant, typically the source-term integration (`W3SRCE`) and spatial propagation, using standard Fortran 2008/2018 constructs (`do concurrent`, explicit interfaces, contiguous arrays, elimination of large automatic arrays) without altering the numerical algorithm. Results must match the reference within a tolerance defined per output field.
4. **GPU feasibility on an NVIDIA H100.** A study, not a port: which kernels could run on the device, what data would have to reside there, and what speed-up is plausible, informed by the published attempts on WW3 and on the WAM model, and by a small prototype of one kernel. The prototype follows the two-stage, literal-translation recipe with a validation ladder documented for FESOM2 (Koldunov et al., 2026): the chosen kernel is first reproduced as a plain C reference checked against the Fortran on captured inputs, then expressed in Kokkos, with the CPU (Serial) backend required to match the C reference bit-for-bit before any GPU number is reported. The outcome is a written recommendation with measurements, not an operational GPU build.

Out of scope are the multi-grid driver (`ww3_multi`), coupling to atmosphere or ocean models, data assimilation, unstructured grids with implicit solvers (PDLIB) unless the operational grid requires them, changes to the physics parameterisations, and any contribution to or dependence on the WAVEWATCH IV project, which is discussed in the justification below.

5. JUSTIFICATION

An operational forecasting chain is only as useful as its timeliness. For a wave forecast that feeds coastal warnings, port operations, offshore activities and the products distributed through ReNOMO, a shorter model cycle means earlier delivery, room for higher resolution or larger domains at the same delivery time, and the possibility of running several members to quantify uncertainty. The computational core of that chain, WW3, was designed for the supercomputers of the last two decades and carries choices, in its build system, its data layout and its parallel decomposition, that are not automatically the best ones for the machines a university laboratory operates today: multi-core nodes with large memory, fast

NVMe storage and, increasingly, GPUs.

The first two rungs of the proposed ladder, build options and run configuration, are justified by experience and by the model’s own documentation: the switch file and the compiler flags decide which code is compiled and how, and the parallel layout decides how much time is spent moving spectra between processes rather than computing them (The WAVEWATCH III Development Group, 2019). These rungs are cheap to try and frequently overlooked in operational setups that were configured once and never revisited. They also produce the instrument every later rung needs: a reproducible benchmark and a profile.

The third rung is justified by the structure of the code. Published profiling of WW3 shows that the source-term integration dominates the run time, with propagation second, and that the routines involved were written with large local arrays and module-level global state that hinder both vectorisation and any form of offloading (Ikuyajolu et al., 2023). Targeted rewriting of these routines in modern Fortran, keeping the algorithm and the arithmetic, is a well-understood engineering task whose value can be measured directly.

The fourth rung addresses the question the laboratory actually asks, whether an H100 could run the model, and answers it honestly. Upstream WW3 has no GPU execution path. The only published port moved the source terms to GPU with OpenACC directives and obtained roughly 1.3 times speed-up per node on the Summit supercomputer, limited by the transfer of data between host and device at every time step (Ikuyajolu et al., 2023). By contrast, the WAM6-GPU project, which refactored the whole model so that every field stays resident on the device, reported an order-of-magnitude gain on a multi-GPU node (Yuan et al., 2024). The lesson is that partial offloading fails and that a real GPU port of WW3 is a rewrite of its data layout, a multi-year effort. This project does not attempt that rewrite; it measures what a single node can do today, prototypes one kernel, and documents the gap. If a portability layer is used for the prototype, Kokkos is the natural candidate, being the layer adopted by the Earth-system codes that have made this transition (Trott et al., 2022).

That a Fortran ocean model of this kind can in fact be carried to C++/Kokkos, and how, is no longer a conjecture. Koldunov et al. (2026) report the port of FE-

SOM2, an unstructured-mesh ocean and sea-ice model of about 74,000 lines of core Fortran, to C and then to C++/Kokkos, performed by an agentic LLM coding assistant directed by domain experts. Three elements of their recipe are the reasons this project can treat the Kokkos path as feasible rather than speculative. First, the translation was done in two stages, Fortran to a clean single-threaded C reference and only then C to Kokkos, so that numerics and parallelisation were never changed at the same time. Second, translation was strictly literal, with algorithmic changes forbidden, which made per-routine comparison against the Fortran possible. Third, validation was a ladder: the C version reproduces the Fortran at the level of five-year simulation statistics, the Kokkos version is bit-for-bit identical to the C on CPU, and statistically close on GPU, where a single A100 node ran 1.6 to 3.7 times faster than a CPU node. The whole translation took weeks, not years. WW3 is a structured-grid spectral model with a per-point source-term kernel that is, if anything, better suited to this recipe than FESOM2’s unstructured dynamics; what the recipe requires, and what this project’s first three rungs produce, is a frozen configuration, a profiled reference and a parity gate for every routine.

A specific justification concerns the relationship with WAVEWATCH IV (WW4), the ground-up rewrite of the model in C++ announced by NOAA/NCEP (NOAA-EMC, 2026b; NOAA/NWS/NCEP, 2025). WW4 is at a pre-alpha stage, with a driver skeleton and no physics, and its first public release is not expected before 2027; NOAA’s own plan states that WW3 support will be sunset only once WW4 matures. For the laboratory this means that the WW3 it runs today will remain its operational model for years. This project therefore does not contribute to WW4, does not fork WW3’s physics, and does not build a competing model. It optimises the operation of the existing model, and every artefact it produces, the benchmark, the profile, the configuration record and the GPU study, is written so that it can inform the WW4 developers rather than duplicate their work. This separation is stated explicitly to keep the scope of an undergraduate project realistic and to avoid effort that a funded team is already spending.

Finally, the project is justified by its reproducibility. All tooling is developed in an open repository that already provides a pinned, declarative toolchain (compilers, MPI, NetCDF and analysis tools from a single package-set revision), scripts to

build the model and run its regression tests, and continuous integration (Hoffmann, 2026). Every measurement in the project can be repeated by the laboratory on its own machines from that repository.

6. OBJECTIVE

The general objective is to reduce, in a measured and reproducible way, the wall-clock time of the operational WAVEWATCH III cycle run by LabECO/UFSC within ReNOMO, without changing its scientific results beyond tolerances agreed with the laboratory, and to deliver a written recommendation on how far the laboratory’s hardware, including an NVIDIA H100, can take that reduction.

The specific objectives are:

1. To freeze and document the operational configuration (source revision, switch file, namelists, grid, forcing, outputs, hardware) and to build a reproducible benchmark of the reference run, with a defined metric (wall-clock per forecast hour and spectral bin updates per second).
2. To produce a profile of the reference run by routine and by phase (source terms, propagation, gather/scatter communication, I/O), on one and on several MPI ranks.
3. To quantify the gain of each rung of the optimisation ladder, build options, run configuration and targeted refactoring, each with its parity evidence against the reference, and to deliver the best configuration to the laboratory as a documented build.
4. To assess the feasibility of GPU execution on an H100 through a study and a single-kernel prototype, reporting measured numbers and the data-movement constraints that bound them.
5. To publish the tooling, the results and the recommendation in the open repository, in a form that the laboratory can rerun and that the WW4 developers can consult.

7. METHODOLOGY

The project follows a measurement-first method. No optimisation is attempted before the reference run is reproducible and profiled, and no optimisation is accepted without a parity check against that reference.

Reference and benchmark. The laboratory’s operational case is captured as a self-contained test: input files, forcing for a fixed period, the switch file and the namelists, and the reference outputs. The same procedure is applied to the upstream regression test closest to the operational configuration, so that results can be compared with the community’s own cases. The benchmark script records wall-clock per phase, the number of ranks and threads, the compiler and flags, and the source revision, and writes a table that later runs append to.

Profiling. The reference run is profiled with `gprof` or `perf` at one, four and sixteen ranks, and communication time is recorded separately. The result is a table of inclusive and exclusive time per routine, grouped into the phases of the time loop. This table decides the order of the refactoring work and is the baseline every later measurement refers to.

Parity gates. For rungs one and two, outputs are compared bit-for-bit; when a compiler flag reorders floating-point operations, the comparison uses a per-field tolerance (for example on significant wave height, peak period and mean direction) proposed by the student and approved by the co-advisor. For rung three, the same tolerances apply, and each rewritten routine is additionally tested in isolation against its original on captured inputs. A change that fails its gate is not merged.

Rungs. Build options are explored first, as a matrix of compiler, flags and parallel layout, on the laboratory’s nodes. Run configuration follows, changing one namelist parameter at a time. Refactoring targets the routines at the top of the profile, one at a time, in a fork of the model that keeps the original routine callable behind a runtime switch, so that both paths can be run on the same binary. The GPU study reviews the published ports, measures the data volume that would cross between host and device per time step in the laboratory’s configuration, and prototypes one kernel (the nonlinear interaction term, which is per-point and self-contained) with a portability layer, reporting its speed-up on the H100 against the CPU version. The prototype is built the way FESOM2 was ported (Koldunov et al., 2026): a literal C translation of the routine, gated against the Fortran on captured inputs; a Kokkos version whose Serial backend must be bit-for-bit identical to the C; and only then the CUDA backend, compared statistically. An LLM coding assistant is used for the translation under the same rules, with the student and the co-advisor

as the domain experts who direct it and own the validation; the prompts and the parity evidence are committed with the code.

Tooling and reproducibility. All work is done in the open `ww-lab` repository (Hoffmann, 2026), which provides the pinned toolchain, the build and regression-test scripts, the model as a submodule pinned to a fixed revision, and continuous integration that checks the scripts and builds the documentation. Results are recorded in the repository as tables and plots generated by scripts, never by hand.

Deliverables and validation. The deliverables are the configuration record, the benchmark and profile tables, the documented best build, the GPU feasibility memo with measurements, and this proposal’s final report. Validation of the scientific side is done with the co-advisor at LabECO, who defines the tolerances and reviews the parity evidence; validation of the engineering side is done through the repository’s automated tests and through independent reruns on the laboratory’s machines.

Risks. The main risk is silent divergence of results after a refactoring, mitigated by the parity gates and by per-routine tests. The second is scope creep towards a full GPU port, mitigated by the explicit limits in the scope section. The third is that the laboratory’s operational grid turns out to be unstructured, in which case rung three is restricted to the source terms and the propagation part is left to the study.

8. SCHEDULE

The project is planned for two academic periods. Dates are proposals to be adjusted with the advisors.

Table 1: Undergraduate project schedule.

Stage	Deadline
Bibliographic review; configuration record of the operational case	10/2026
Reproducible benchmark and profile of the reference run	11/2026

Stage	Deadline
Rung 1: build options (compiler, flags, switch file, parallel layout)	12/2026
Rung 2: run configuration (decomposition, time steps, output)	01/2027
Rung 3: targeted modern-Fortran refactoring of the profiled hotspots	02/2027
Rung 4: GPU feasibility study and single-kernel prototype on the H100	03/2027
Written recommendation to LabECO; final report and revision	04/2027
Defence	from 05/2027

References

HOFFMANN, Matheus. **Ww-lab: A bootstrap repository for WAVEWATCH III**. <https://github.com/h0ffmann/ww-lab>, 2026.

IKUYAJOLU, Olawale James *et al.* [Porting the WAVEWATCH III \(v6.07\) wave action source terms to GPU](#). *Geoscientific Model Development*, v. 16, p. 1445–1458, 2023.

KOLDUNOV, Nikolay V. *et al.* [An ocean model ported by a large language model: Experience and lessons from FESOM2 \(Fortran to C to C++/Kokkos\)](#). arXiv:2606.11356, 2026.

LABECO/UFSC. **ReNOMO – rede nacional de observação e monitoramento oceânico**. <https://labeco.ufsc.br/renomo/>, 2026.

NOAA-EMC. **WAVEWATCH III source code repository**. <https://github.com/NOAA-EMC/WW3>, a2026.

NOAA-EMC. **WAVEWATCH IV (WW4) repository**. <https://github.com/>

NOAA-EMC/WW4, b2026.

NOAA/NWS/NCEP. **WAVEWATCH IV: Plan for the next-generation wave model**. *[S.l.]*: NCEP, 2025.

THE WAVEWATCH III DEVELOPMENT GROUP. **User manual and system documentation of WAVEWATCH III version 6.07**. College Park, MD, USA: NOAA/NWS/NCEP/MMAB, 2019.

TROTT, Christian R. *et al.* **Kokkos 3: Programming model extensions for the exascale era**. *IEEE Transactions on Parallel and Distributed Systems*, v. 33, n. 4, p. 805–817, 2022.

UFSC. **Laboratório de engenharia e ciências oceânicas (LabECO)**. <https://emc.ufsc.br/portal/laboratorios-2/labeco/>, 2026.

YUAN, Ye *et al.* **WAM6-GPU v1.0: GPU-accelerated third-generation wave model**. *Geoscientific Model Development*, v. 17, p. 6123–6136, 2024.

Rio de Janeiro, September 13, 2026.

Matheus Hoffmann Fernandes Santos - Student

To be defined, Dr. - Advisor

Pedro Veras Guimarães, Dr. - Co-advisor